TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING

Khwopa College Of Engineering
Libali, Bhaktapur
Department of Computer Engineering

A FINAL REPORT ON

STUDY APP: STUDY BUG

Submitted in partial fulfillment of the requirements for the degree

BACHELOR OF COMPUTER ENGINEERING

Submitted by

Anjana Silinchhe Shrestha	KCE080BCT004
Pradipta Joshi	KCE080BCT021
Prasant Rai	KCE080BCT024
Sushma Shrestha	KCE080BCT043

Khwopa College Of Engineering
Libali, Bhaktapur
March 2026

Certificate of Approval

This is to certify that the project entitled **"StudyBug"** submitted by Anjana Silinchhe Shrestha, Pradipta Joshi, Prasant Rai and Sushma Shrestha to the **Department of Computer and Electronics Engineering** as a **group project**, in partial fullfillment of the requirements for the award of the degree/course in Computer Engineering. The project was carried out under special supervision and within the time frame prescribed by the syllabus.

We found the students to be hardworking, skilled, bona fide and ready to undertake any commercial and industrial work related to their field of study and hence we recommend the award of Bachelor of Computer Engineering degree.

.....

Er. Sunil Banmala
Supervisor
Department of Computer
and Electronics
Engineering, KhCE

.....

Er. Ramesh Prajapati
Supervisor
Department of Computer
and Electronics
Engineering, KhCE

.....

Er. Dinesh Gothe
Head of Department
Department of Computer
and Electronics
Engineering, KhCE

Copyright

Acknowledgement

We take this opportunity to express our deepest and sincere gratitude to our HoD Er. Dinesh Gothe, for his insightful advice, motivating suggestions for this project and also for his constant encouragement and advice throughout our Bachelor's program.
We are deeply indebted to our Supervisor Er. Sunil Banmala for boosting our efforts and moral by his valuable advises and suggestion regarding the project, giving us realization of the practical scenario of the project and supporting us in tackling various difficulties.

Anjana Silinchhe Shrestha	KCE080BCT004
Pradipta Joshi	KCE080BCT021
Prasant Rai	KCE080BCT024
Sushma Shrestha	KCE080BCT043

Abstract

StudyBug is a comprehensive, full-stack productivity and study management web application developed to address common challenges faced by students such as lack of focus, poor time management, and inconsistent study habits. The system integrates essential study-support features including task management, study scheduling, Pomodoro-based focus timers, progress tracking, quiz generation from uploaded documents, notes and journal modules, and learning gamification within a unified, user-friendly interface. Special emphasis was placed on aesthetic design to create a calm and engaging virtual study environment that reduces distractions and increases motivation. The application was developed using React 19 for the frontend, Node.js with Express.js for the backend, and PostgreSQL for data persistence, with Supabase handling authentication. Over an 8-week development cycle, the team successfully implemented 13 major features, 35+ API endpoints, and 12 database tables. The website is fully responsive and accessible across both desktop and tablet devices, allowing users to study flexibly. By combining productivity tools with thoughtful UI/UX design and gamification elements (XP, streaks, levels), StudyBug demonstrates how technology can effectively improve study consistency, enhance concentration, and support effective learning practices.

Keywords: *Study website, Focus timer, Pomodoro technique, Task management, Quiz generation, Progress tracking, Learning gamification, React, Node.js, Full-stack web application*

Contents

List of Figures

List of Abbreviation

Abbreviations	Meaning
UI	User Interface
UX	User Experience
API	Application Programming Interface
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
JWT	JSON Web Token
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
SQL	Structured Query Language
DFD	Data Flow Diagram
ER	Entity Relationship
XP	Experience Points
PDF	Portable Document Format
CSV	Comma Separated Values
SPA	Single Page Application

Chapter 1

Introduction

1.1 Background Introduction

In today's academic environment, students are expected to manage heavy workloads, tight deadlines, and continuous assessments. Effective studying requires concentration, proper planning, and consistency. However, many students struggle to maintain focus during study sessions due to distractions such as mobile phones, social media, and mental fatigue. In addition to focus-related issues, the absence of a structured study schedule often leads to procrastination, stress, and inefficient learning.

Although several digital tools such as calendars, to-do lists, and focus timers exist, most of them are either too complex or lack long-term engagement. Many study applications focus solely on functionality and ignore the emotional and visual aspects that influence motivation. As a result, students often abandon these tools after short-term use.

To address these challenges, this project developed StudyBug—a comprehensive, full-stack productivity and study management web application that combines focus enhancement, structured scheduling, knowledge assessment through quizzes, and an engaging visual experience. The goal was to create a study environment that not only improves productivity but also makes studying more enjoyable and sustainable through gamification elements.

1.2 Motivation

The motivation for this project originates from our personal experience as students. While studying, we frequently faced loss of focus during study sessions, lack of a proper and consistent study schedule, and reduced motivation because studying felt monotonous and stressful.
We observed that even when study resources were available, the absence of an engaging and organized system made it difficult to stay disciplined. Traditional study methods and existing apps felt repetitive and uninteresting, which further reduced consistency.
This realization led to the idea of developing a study application that is visually

aesthetic, structured, and enjoyable to use. By making the study process more appealing and organized, combined with gamification elements like XP points, streaks, and levels, the application helps students stay focused, manage time better, and develop healthier study habits.

1.3 Problem Definition

Despite the availability of numerous productivity and study-related applications, many students continue to struggle with maintaining focus and consistency in their academic routines. Common challenges include:

- Frequent distractions during study sessions
- Ineffective time management due to a lack of structured scheduling
- Low engagement with existing tools that fail to provide visual or emotional motivation
- Difficulty tracking learning progress over time
- Lack of self-assessment tools for knowledge retention

These issues highlight the need for an integrated study application that not only supports focused study and effective planning but also enhances motivation through an aesthetic, intuitive, and user-friendly interface with gamification elements.

1.4 Goals and Objectives

The main objectives of this project were:

- To analyze common problems faced by students during studying
- To develop a comprehensive web-based study platform with task management, scheduling, and focus timer features
- To implement a gamification system (XP, streaks, levels) to maintain user engagement
- To create an automatic quiz generation system from uploaded documents
- To provide progress tracking and analytics for study sessions
- To design an aesthetic and distraction-minimized user interface

1.5 Scope and Applications

The major scope of the project was to assist students in developing effective study habits. The developed system provides:

- Study scheduling and task planning with priority levels

- Pomodoro-based focus timer functionality for structured study sessions
- Progress tracking with gamification (XP points, streaks, 15 learning levels)
- Automatic quiz generation from uploaded PDF and TXT documents
- Notes module with image support
- Daily journal with mood tracking
- Calendar with multi-day events and task synchronization
- Focus timeline analytics (daily, weekly, monthly views)
- Export functionality for progress reports (CSV and PDF)
- Aesthetic UI design with virtual study room environment

1.6 Report Organization

1.6.1 Introduction

The introduction sets the scene for readers to understand the problems that students face during studying and how the StudyBug system addresses them.

1.6.2 Literature Review

The literature review familiarizes with the current state of knowledge on study productivity tools and ensures the solution addresses gaps not covered by existing applications.

1.6.3 Feasibility Study

It determines the viability of the project, ensuring it is technically feasible, operationally practical, and economically justifiable.

1.6.4 Methodology

It critically analyzes and describes the development methodology used for the project.

1.6.5 Requirement Analysis

It provides a clear vision about the hardware, software, functional, and non-functional requirements of the StudyBug web application.

1.6.6 System Design and Architecture

The main purpose is to evaluate the contribution of each component for overall performance of the system using different diagrams.

1.6.7 Implementation and Development

This chapter describes the actual implementation of the system, including the technology stack, key features developed, and development progress.

1.6.8 Testing and Results

It presents the testing methodologies used and the actual outcomes achieved by the system.

1.6.9 Conclusion and Future Enhancements

The conclusion summarizes the project achievements and future enhancements provides scope for upgrading the project.

Chapter 2

Literature Review

2.1 Assessing the efficacy of the Pomodoro technique in enhancing anatomy lesson retention during study sessions: a scoping review

In this paper, [**?**] presented a scoping review analyzing the impact of the Pomodoro Technique (PT) on cognitive performance and retention, specifically within the context of anatomy education. The author highlighted that anatomy requires substantial cognitive effort, often leading to mental fatigue when students rely on self-paced study habits. The review found that structured Pomodoro intervals (typically 25 minutes of work followed by 5-minute breaks) were significantly more effective than unstructured, self-regulated breaks.

The study reported that students using the Pomodoro technique experienced approximately 20% lower fatigue levels and a 15–25% increase in self-rated focus compared to control groups. Furthermore, the use of digital tools and timers to enforce these intervals was found to enhance student engagement by 10–18%. The authors concluded that time-structured interventions consistently outperformed self-paced study sessions by reducing distractibility and sustaining motivation over longer periods. This suggests that integrating Pomodoro timers into productivity applications can serve as a critical mechanism for preventing cognitive overload and improving long-term retention of complex material.

Application to StudyBug: Based on these findings, StudyBug implemented a fully functional Pomodoro timer with configurable duration, automatic break reminders, and session tracking. The system associates focus sessions with specific tasks and tracks progress percentage, enabling users to benefit from the structured time management approach proven effective in the literature.

2.2 Analyzing the Impact of AI Tools on Student Study Habits and Academic Performance

The integration of mobile technology in higher education has created new opportunities for enhancing student productivity and self-regulation. [**?**] conducted a comprehensive survey involving 269 academic staff and higher-degree students, revealing that while nearly 95% of students possess smartphones, the use of mo-

bile applications for academic purposes is largely driven by personal motivation rather than institutional requirements. The study found that current app usage is predominantly focused on basic document storage and communication tools such as Dropbox, rather than specialized academic process-management applications.

However, a significant gap exists between current usage patterns and student needs. [**?**] reported that students specifically recommended applications for project and assignment planning, and non-users expressed a strong willingness to adopt academic apps if they were easier to use and more appropriate for their academic context. These findings indicate a clear demand for simplified, student-centric productivity applications that extend beyond basic file storage.

Application to StudyBug: StudyBug addresses this gap by providing an integrated platform that combines task management, scheduling, focus sessions, notes, and journal features. The application was designed with an intuitive interface to lower the barrier to adoption and includes file management capabilities that enable quiz generation from uploaded documents.

2.3 A Study of Mobile App Use for Teaching and Research in Higher Education

In a comprehensive study on the integration of mobile technology in universities, [**?**] surveyed 269 academic staff and higher-degree students to examine how mobile applications are utilized in academic settings. The study established that the hardware barrier to mobile learning is virtually non-existent, with recent data indicating that approximately 95% of students possess smartphones. Despite this widespread availability, the authors found that mobile app usage is primarily driven by *personal motivation* rather than institutional planning.

The study further revealed that the current academic app ecosystem is dominated by basic utility tools. Applications for document storage, such as Dropbox, and communication were reported as the most frequently used, whereas specialized study or process-management tools were comparatively underutilized. When participants were asked to recommend purposes for academic app usage, *project and assignment planning* emerged as a key category. Additionally, among participants who did not currently use mobile apps for academic purposes, a significant proportion expressed an intention to adopt such tools in the future, particularly for project or assignment planning and note-taking.

Application to StudyBug: These findings directly informed the feature set of StudyBug. The application provides comprehensive project and assignment planning through its task management system, calendar integration, and notes module. The responsive design ensures accessibility across devices, addressing the cross-platform requirements identified in the research.

2.4 Gamification in Education

Research on gamification in educational contexts has demonstrated that incorporating game-like elements such as points, levels, and streaks can significantly improve student engagement and motivation. Studies have shown that gamifi-

cation can increase time spent on learning activities by up to 40% and improve completion rates of educational tasks.

The psychological principles underlying gamification include:

- **Progress visualization**: Showing users their advancement through levels provides a sense of achievement
- **Streak mechanics**: Daily streak counters create commitment and habit formation
- **Experience points**: XP systems provide immediate feedback and reward for completed activities

Application to StudyBug: StudyBug implements a comprehensive gamification system with:

- 3 learning phases with 5 levels each (15 total levels)
- XP points earned through completing quizzes and tasks
- Daily streak tracking to encourage consistent study habits
- Productivity score calculation based on completed activities

2.5 Summary of Literature Review

The literature review reveals several key insights that guided the development of StudyBug:

1. The Pomodoro Technique is scientifically validated to improve focus and reduce cognitive fatigue
2. Students desire integrated productivity applications but current options are either too complex or lack engagement
3. Gamification elements significantly improve user engagement and habit formation
4. There is a gap in the market for student-centric applications that combine planning, focus management, and progress tracking

StudyBug was designed to address these findings by providing a comprehensive, engaging, and user-friendly study management platform that incorporates evidence-based techniques for improving student productivity.

Chapter 3

Feasibility Study

3.1 Technical Feasibility

The StudyBug web application was technically feasible as all required features were implemented using standard web technologies. The system was developed using:

- **Frontend**: React 19 with React Router DOM 7 for client-side routing
- **Backend**: Node.js with Express.js 4.18 for RESTful API development
- **Database**: PostgreSQL for persistent data storage
- **Authentication**: Supabase Auth with JWT tokens for secure user management
- **File Processing**: pdf.js (pdfjs-dist) for PDF parsing in quiz generation

All technologies used are mature, well-documented, and widely adopted in the industry. The responsive design ensures the system works across desktop and tablet devices through any modern web browser.

3.1.1 Technology Assessment

Technology	Maturity	Availability	Assessment
React 19	High	Open Source	Excellent
Node.js	High	Open Source	Excellent
Express.js	High	Open Source	Excellent
PostgreSQL	High	Open Source	Excellent
Supabase	High	Free Tier	Excellent

Table 3.1: Technology Assessment Matrix

3.2 Operational Feasibility

Operational feasibility assessed how well the system works in real conditions and how easily users can adopt it.

The developed website is operationally feasible because it successfully addresses real student challenges: lack of focus, unstructured schedules, and low motivation. The system provides a simple and practical solution—users can plan tasks, schedule study time, run focus sessions, and track progress within one platform. Since the website is accessible through any browser, users do not need installation, which improves accessibility and adoption.

3.2.1 Focus Timer and Study Session Module

This module successfully supports focused study techniques using the Pomodoro method. Users can start and pause study sessions, take short breaks, and record study time. The distraction-minimized focus view improves usability. Session tracking and task association work reliably in the browser environment.

3.2.2 Study Scheduling and Planner Module

The scheduling module allows users to create daily or weekly study plans and allocate time blocks for tasks. This improves structure and reduces procrastination. The calendar interface presents schedules in a clean format with multi-day event support and drag-and-drop functionality.

3.2.3 Aesthetic UI, Themes, and Personalization

This module focuses on the visual appeal through an immersive virtual study room with interactive hotspots. The aesthetic design improves engagement and increases long-term use by making the study environment more pleasant. The video background and interactive elements support the core idea that studying should feel less stressful.

3.2.4 Quiz Generation Module

The quiz generation module successfully extracts text from uploaded PDF and TXT files, analyzes word frequency, identifies key terms, and generates fill-in-the-blank MCQ questions. This feature adds significant value by enabling self-assessment.

3.3 Economic Feasibility

The project was economically feasible because it was developed using free and open-source tools. The actual costs incurred were minimal:

The project successfully demonstrated that a comprehensive study application can be developed and deployed with minimal financial investment, making it suitable for academic projects and potential future scaling.

Cost Category	Amount	Notes
Development Tools	$0	VS Code, Git (Free)
Frontend Hosting	$0-5/month	Vercel/Netlify free tier
Backend Hosting	$0-7/month	Railway/Render free tier
Database (Supabase)	$0/month	Free tier sufficient
Domain (Optional)	$10-15/year	Optional custom domain
Total Monthly Cost	**$0-12/month**	Scalable as needed

Table 3.2: Economic Feasibility - Cost Analysis

3.4 Schedule Feasibility

The StudyBug project was completed within an 8-week development cycle, demonstrating schedule feasibility. The project followed an iterative Agile approach:

Phase	Duration	Status
Project Setup & Planning	Week 1-2	Completed
Core Backend Development	Week 2-3	Completed
Frontend-Backend Integration	Week 3-4	Completed
Feature Development	Week 4-6	Completed
Advanced Features	Week 6-7	Completed
Polish & Documentation	Week 7-8	Completed
Total	**8 weeks**	**Completed**

Table 3.3: Project Schedule - Actual Timeline

The modular approach and clear separation of frontend and backend development allowed the team to work efficiently and meet all deadlines without schedule overruns.

Chapter 4

Methodology

4.1 Agile Method as Software Development Model

The development of StudyBug followed the Agile software development methodology. Agile was chosen because the project emphasized user experience, iterative design, and continuous improvement. Since the application includes multiple interactive features such as scheduling, focus timers, task management, quiz generation, and aesthetic customization, Agile allowed these components to be developed and refined incrementally.

The Agile approach enabled frequent evaluation of progress, early detection of issues, and flexibility in incorporating feedback. Each iteration (sprint) focused on delivering a functional and usable version of the application, which was then enhanced in subsequent iterations.

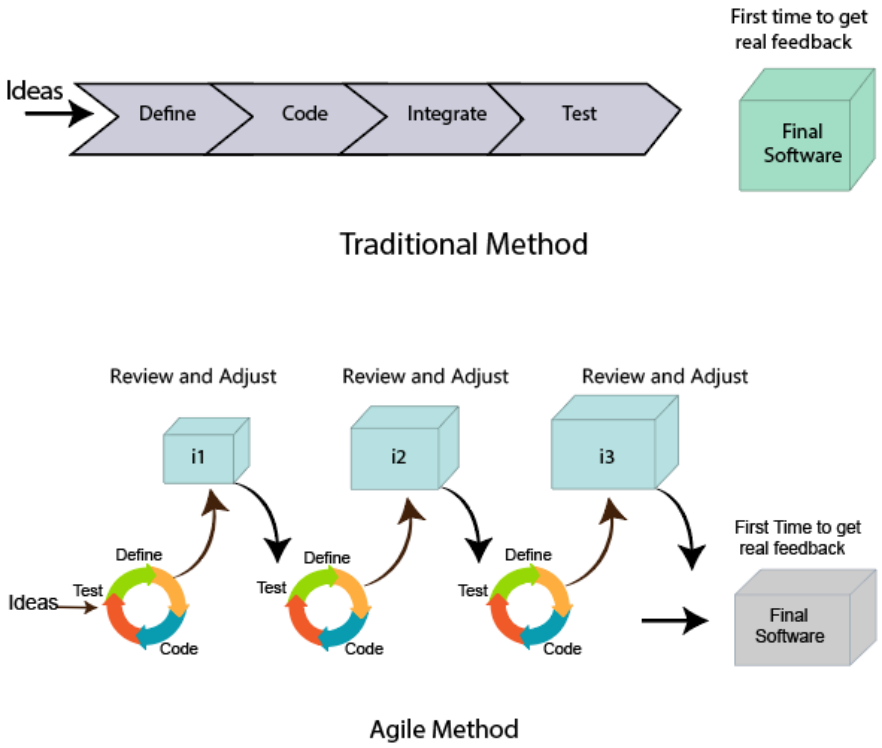


Figure 4.1: Agile Model as Software Development Model

4.2 Development Phases

The project was completed in the following phases over 8 weeks:

4.2.1 Week 1-2: Project Initialization and Setup

- React application scaffold created using Create React App
- Basic project structure established
- StudyRoom with clickable hotspots implemented
- Express.js backend server set up
- Repository structure organized (frontend/backend separation)

4.2.2 Week 3: Frontend-Backend Integration

- Frontend connected to backend API
- Supabase Auth integration completed
- User-specific data isolation implemented
- Security fixes applied (users can only see their own data)
- First successful pull request merged

4.2.3 Week 4: Calendar and Task Management

- Notion-style calendar implemented with monthly view
- Task-Event differentiation created
- Bi-directional sync between Todo and Calendar
- UI styling improvements completed
- Loading screen redesigned

4.2.4 Week 5: Major Feature Integration

- Quiz system with document-based question generation
- Dashboard with learning progress tracker (3 phases, 15 levels)
- Notes module with rich text support
- Journal with mood tracking
- Pomodoro timer initial implementation
- XP and streak tracking system

4.2.5 Week 6: Pomodoro Refinement

- Complete Pomodoro timer functionality
- Task synchronization with focus sessions
- Session progress tracking
- Break timer implementation
- Pause/resume functionality

4.2.6 Week 7: Advanced Features

- File-based quiz generation (PDF/TXT support)
- Focus timeline with daily/weekly/monthly views
- Export functionality (CSV and PDF)
- Image support in notes (up to 5 images)
- Enhanced productivity calculation

4.2.7 Week 8: Polish and Documentation

- UI/UX polish and improvements
- Drag-and-drop task rescheduling
- Multi-day event support
- Comprehensive documentation updates
- Final bug fixes

4.3 Figma for Ideation and Prototyping

Figma was used for ideation and prototyping to design wireframes and interactive layouts of the StudyBug website. It helped visualize user flows, refine the interface, and ensure an aesthetic and user-friendly design before development.

4.4 Version Control and Collaboration

GitHub was used for version control and collaborative development. The team followed a structured branching strategy:

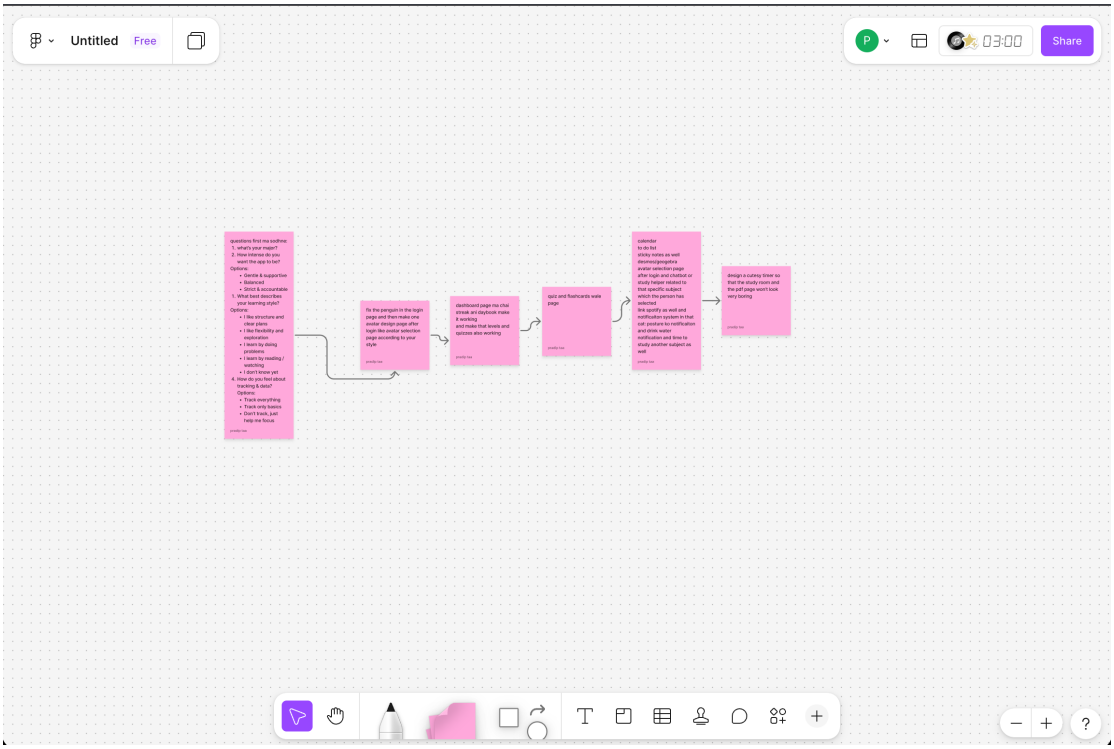


Figure 4.2: Figma Workflow

4.4.1 Branch Structure

- **main**: Production-ready code
- **sushma/backend**: Backend development branch
- **front/anjana**: Frontend development branch
- **sushma/frontend**: Frontend refinements
- **pomodoro**: Pomodoro timer features
- **fix/functionalities**: Bug fixes
- **documentation**: Documentation updates

4.4.2 Pull Request Workflow

1. Team members worked on assigned tasks using separate branches
2. Changes were implemented locally and pushed to GitHub
3. Pull requests were created for code review
4. Completed work was reviewed and merged into the main branch

A total of 13 pull requests were successfully merged during development, with 52 commits tracking incremental progress.

4.5 Task Workflow

Each task was managed and recorded procedurally:

1. Tasks assigned based on team agreement
2. Implementation on feature branches
3. Local testing before pushing
4. Code review through pull requests
5. Merge to main branch after approval
6. Integration testing after merge

4.6 Commit Statistics

The development produced the following commit distribution:

Commit Type	**Count**	**Percentage**
Feature (feat:)	6	11.5%
Fix (fix:)	17	32.7%
Refactor	7	13.5%
Merge	8	15.4%
Update	14	26.9%
Total	**52**	**100%**

Table 4.1: Commit Type Distribution

Chapter 5

Requirement Analysis

5.1 Hardware Requirements

5.1.1 Development Environment

Component	Minimum	Recommended
Processor	Intel i3 / AMD Ryzen 3	Intel i5+ / AMD Ryzen 5+
RAM	8 GB	16 GB
Storage	20 GB free	50 GB+ SSD
Display	1366 x 768	1920 x 1080+
Internet	5 Mbps	25 Mbps+

Table 5.1: Development Environment Hardware Requirements

5.1.2 Server/Hosting Requirements

Component	Minimum	Notes
Node.js Runtime	v16+	v18+ recommended
Memory	512 MB	For Express server
Storage	1 GB	For file uploads
Database	Managed (Supabase)	Free tier: 500 MB

Table 5.2: Server/Hosting Hardware Requirements

Component	Requirement
Browser	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
JavaScript	Enabled
Cookies	Enabled (for session management)
Screen	Desktop and tablet supported

Table 5.3: Client Hardware Requirements

Software	Version	Purpose
React	19.x	UI Framework
React DOM	19.x	React rendering for web
React Router DOM	7.x	Client-side routing
Supabase JS	2.39+	Authentication client
React Scripts	5.x	Build tooling (CRA)

Table 5.4: Frontend Software Stack

5.1.3 Client Requirements

5.2 Software Requirements

5.2.1 Frontend Software Stack

5.2.2 Backend Software Stack

5.3 Functional Requirements

5.3.1 FR-1: User Authentication

5.3.2 FR-2: Task Management

5.3.3 FR-3: Focus Sessions (Pomodoro)

5.3.4 FR-4: Calendar Management

5.3.5 FR-5: Notes and Journal

5.3.6 FR-6: Quiz Generation

5.3.7 FR-7: Progress Tracking and Reporting

5.4 Non-Functional Requirements

5.4.1 NFR-1: Performance

5.4.2 NFR-2: Security

5.4.3 NFR-3: Usability and Maintainability

Software	Version	Purpose
Node.js	16+	Runtime environment
Express.js	4.18.x	Web framework
PostgreSQL	14+	Primary database
pg (node-postgres)	8.x	PostgreSQL client
Supabase JS	2.39+	Auth service integration
pdf.js (pdfjs-dist)	3.x	PDF parsing
JWT	9.x	Token handling
Helmet	7.x	Security headers
Morgan	1.x	Request logging
CORS	2.x	Cross-origin support
dotenv	16.x	Environment variables

Table 5.5: Backend Software Stack

ID	Requirement	Priority	Status
FR-1.1	Users shall be able to register with email and password	High	Complete
FR-1.2	Users shall be able to login with registered credentials	High	Complete
FR-1.3	Users shall be able to logout from the application	High	Complete
FR-1.4	Sessions shall persist across browser refreshes	High	Complete
FR-1.5	Sessions shall expire after token validity period	Medium	Complete
FR-1.6	Users shall be able to update their profile	Medium	Complete

Table 5.6: User Authentication Requirements

ID	Requirement	Priority	Status
FR-2.1	Users shall create tasks with title, description, priority	High	Complete
FR-2.2	Users shall set target dates for tasks	High	Complete
FR-2.3	Users shall update task details	High	Complete
FR-2.4	Users shall mark tasks as complete	High	Complete
FR-2.5	Users shall delete tasks	High	Complete
FR-2.6	Tasks shall sync with calendar view	Medium	Complete
FR-2.7	Users shall set estimated pomodoros	Medium	Complete
FR-2.8	Task progress shall update during focus sessions	Medium	Complete

Table 5.7: Task Management Requirements

ID	Requirement	Priority	Status
FR-3.1	Users shall start focus sessions with configurable duration	High	Complete
FR-3.2	System shall track focus session duration	High	Complete
FR-3.3	Users shall pause and resume sessions	Medium	Complete
FR-3.4	Users shall end sessions early	Medium	Complete
FR-3.5	System shall provide break reminders after sessions	Medium	Complete
FR-3.6	Sessions shall be associated with specific tasks	Medium	Complete
FR-3.7	Session history shall be viewable	Medium	Complete

Table 5.8: Focus Sessions Requirements

ID	Requirement	Priority	Status
FR-4.1	Users shall view calendar in monthly format	High	Complete
FR-4.2	Users shall create events	High	Complete
FR-4.3	Users shall create multi-day events	Medium	Complete
FR-4.4	Events shall have types (Exam, Deadline, etc.)	Medium	Complete
FR-4.5	Users shall drag and drop events	Low	Complete
FR-4.6	Tasks shall appear on calendar	Medium	Complete

Table 5.9: Calendar Management Requirements

ID	Requirement	Priority	Status
FR-5.1	Users shall create notes	High	Complete
FR-5.2	Users shall add images to notes (max 5)	Medium	Complete
FR-5.3	Users shall search notes	Medium	Complete
FR-5.4	Users shall create one journal entry per day	High	Complete
FR-5.5	Users shall select mood for journal entries	Medium	Complete

Table 5.10: Notes and Journal Requirements

ID	Requirement	Priority	Status
FR-6.1	System shall generate MCQs from uploaded documents	High	Complete
FR-6.2	Quiz shall have 5 questions	Medium	Complete
FR-6.3	Questions shall be fill-in-the-blank format	Medium	Complete
FR-6.4	Users shall select file for quiz generation	High	Complete
FR-6.5	Default question bank shall be available	Medium	Complete

Table 5.11: Quiz Generation Requirements

ID	Requirement	Priority	Status
FR-7.1	System shall track XP points	High	Complete
FR-7.2	System shall track daily streaks	High	Complete
FR-7.3	System shall calculate productivity score	Medium	Complete
FR-7.4	System shall track learning levels (15 total)	High	Complete
FR-7.5	Users shall export data as CSV	Medium	Complete
FR-7.6	Users shall export data as PDF	Medium	Complete

Table 5.12: Progress Tracking and Reporting Requirements

ID	Requirement	Target
NFR-1.1	Page load time	¡ 3 seconds
NFR-1.2	API response time	¡ 500ms
NFR-1.3	Time to interactive	¡ 5 seconds
NFR-1.4	Database query time	¡ 100ms
NFR-1.5	File upload limit	50 MB

Table 5.13: Performance Requirements

ID	Requirement	Implementation
NFR-2.1	Secure authentication	Supabase Auth with JWT
NFR-2.2	Password encryption	Handled by Supabase
NFR-2.3	HTTPS communication	Enforced in production
NFR-2.4	Input validation	Server-side validation
NFR-2.5	XSS protection	React's built-in escaping
NFR-2.6	SQL injection prevention	Parameterized queries (pg)

Table 5.14: Security Requirements

ID	Requirement	Implementation
NFR-3.1	Intuitive interface	Clean UI with consistent design
NFR-3.2	Responsive design	Desktop and tablet support
NFR-3.3	Error messages	User-friendly error display
NFR-3.4	Modular architecture	Frontend/backend separation
NFR-3.5	Code documentation	Inline comments, README files
NFR-3.6	Version control	Git with meaningful commits

Table 5.15: Usability and Maintainability Requirements

Chapter 6

System Design and Architecture

6.1 High-Level Architecture

The StudyBug application follows a three-tier client-server architecture consisting of:

- **Presentation Layer**: React-based Single Page Application (SPA)
- **Application Layer**: Node.js/Express.js RESTful API server
- **Data Layer**: PostgreSQL database with Supabase authentication

6.2 Use Case Diagram

The Use Case Diagram shows the interactions between the student (primary user) and the system functionalities.

The primary actor (Student) can perform the following use cases:

- Register and Login
- Manage Tasks (Create, Update, Delete, Complete)
- Schedule Study Sessions
- Run Focus Timer (Pomodoro)
- Take Quizzes
- Write Notes and Journal Entries
- View Progress and Analytics
- Export Reports

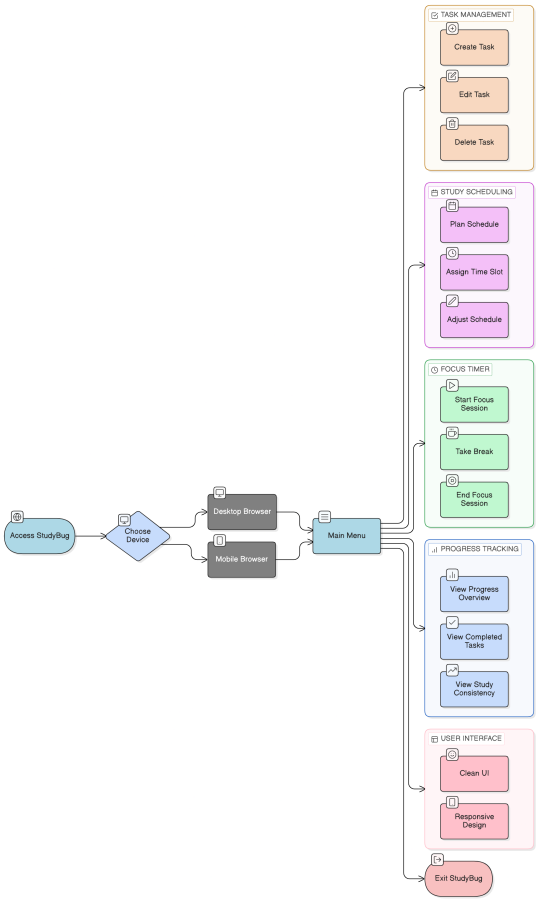


Figure 6.1: Use Case Diagram of StudyBug

6.3 Context Diagram

The Context Diagram shows the top level view of the StudyBug system and its interaction with external entities.

6.4 Data Flow Diagram

The Data Flow Diagram shows the flow of data between the subsystems of StudyBug.

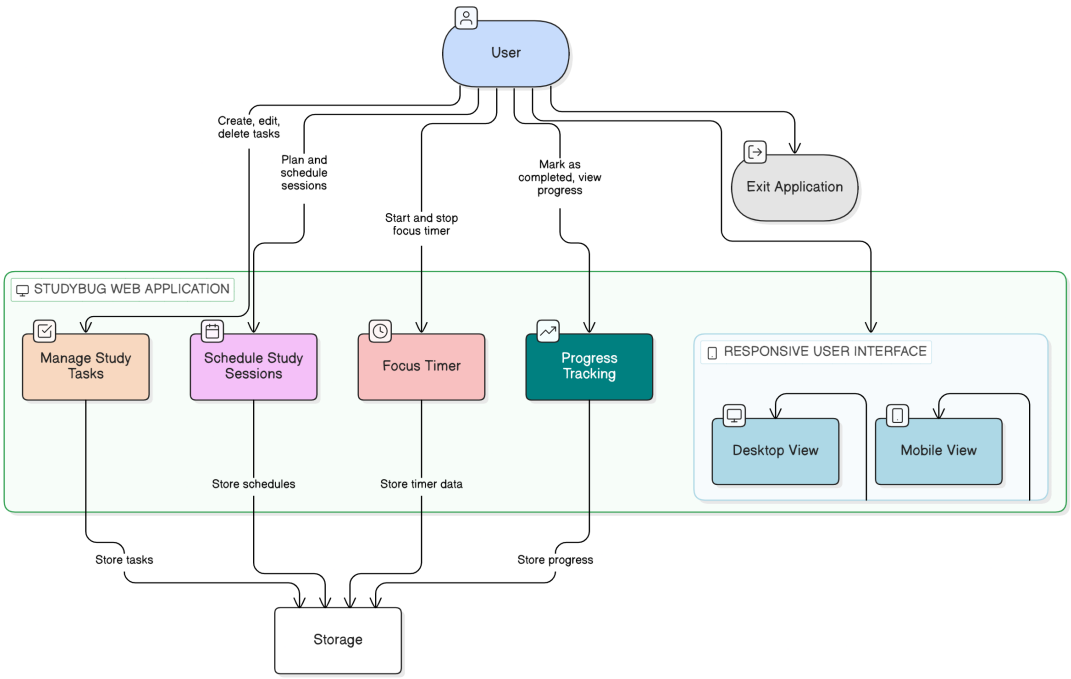


Figure 6.2: Context Diagram of StudyBug

6.5 Sequence Diagram

The Sequence Diagram illustrates the flow of a typical focus session workflow.

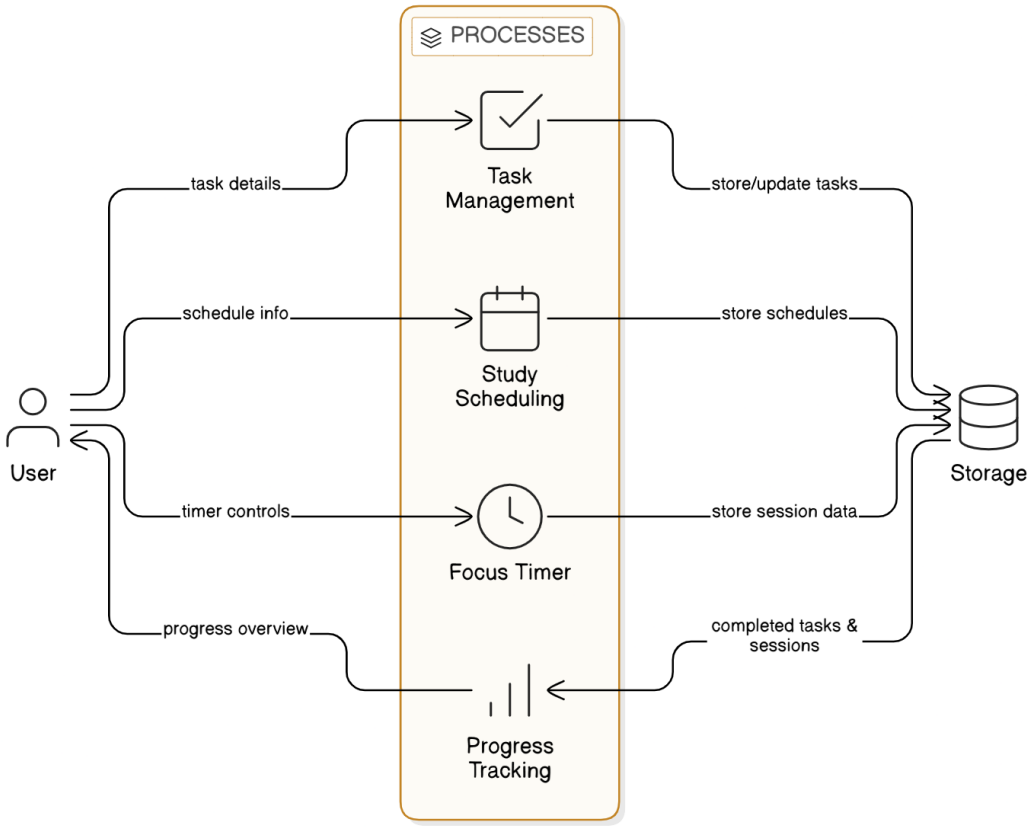
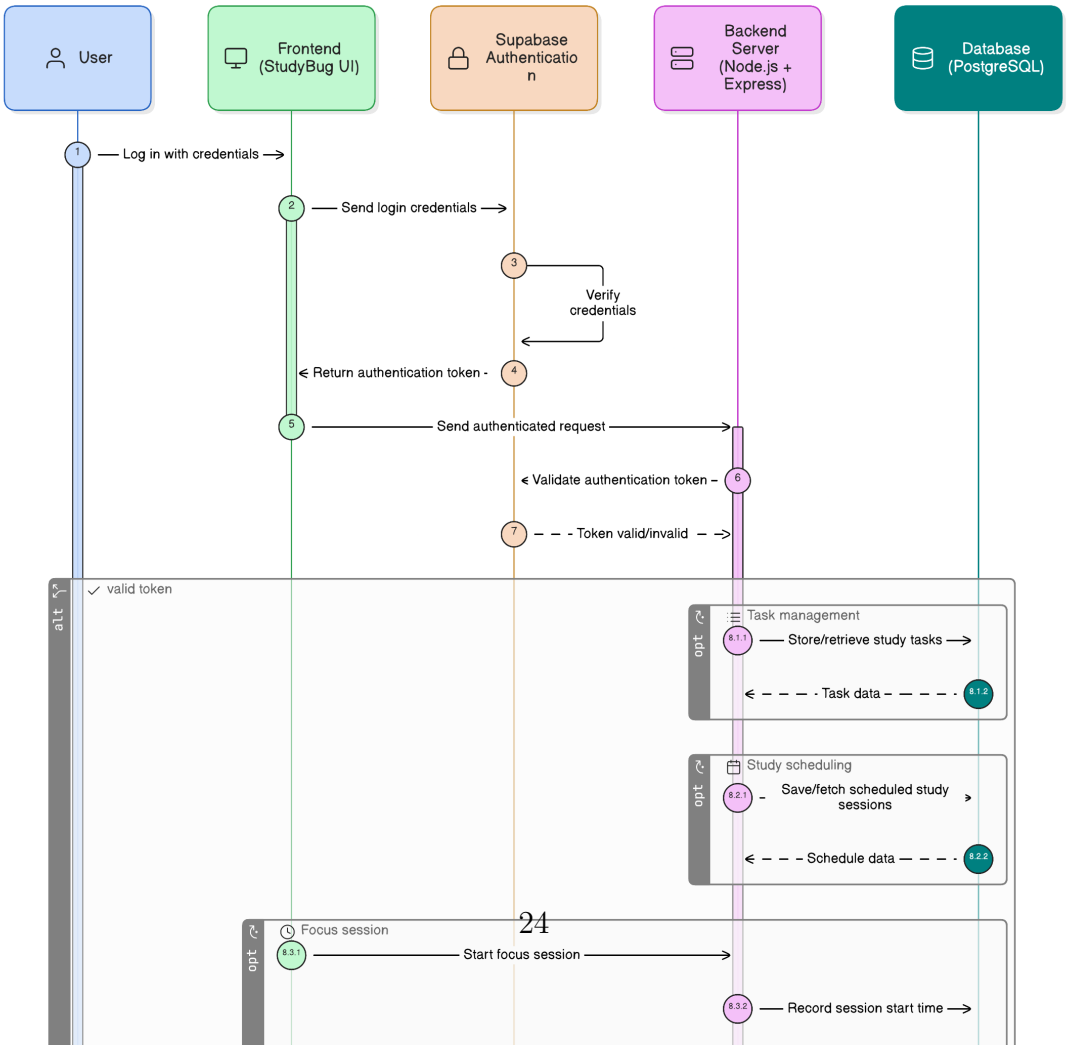


Figure 6.3: Data Flow Diagram of StudyBug



6.6 System Block Diagram

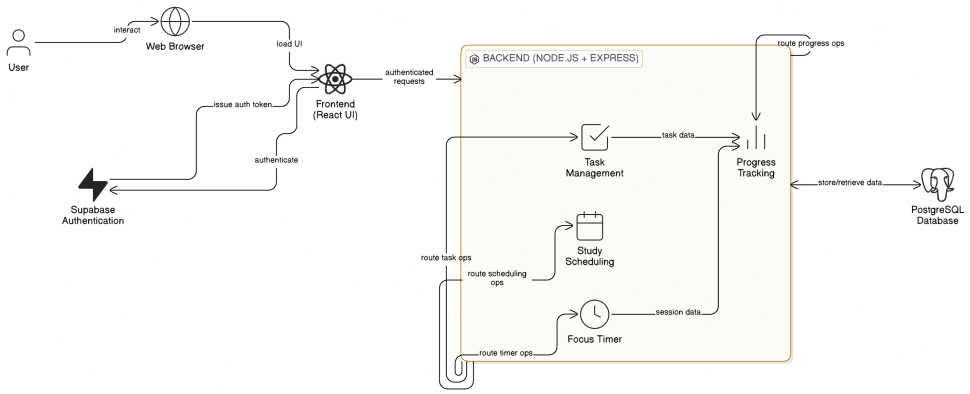


Figure 6.5: System Block Diagram of StudyBug

The system block diagram illustrates the architecture of StudyBug showing how different modules interact:

6.6.1 Frontend Interface Module

The frontend interface module handles user interaction and visual presentation. Developed using React 19, it provides interfaces for:

- Dashboard with learning progress
- Virtual Study Room with interactive hotspots
- Task management and calendar
- Notes and journal pages
- Quiz interface
- Focus timeline analytics

6.6.2 Authentication Module

The authentication module manages user login and access control using Supabase Auth. It verifies credentials and issues JWT tokens for secure communication between frontend and backend.

6.6.3 Backend Server Module

The Express.js backend server acts as the central control unit, handling:

- RESTful API endpoints (35+ endpoints)
- Request validation and error handling
- Business logic processing
- Database communication

6.6.4 Database Module

PostgreSQL database stores all persistent data across 12 tables:

- users, tasks, notes, journal_entries
- calendar_events, files, focus_sessions
- learning_sections, learning_levels
- progress, trash, schedules

6.7 Entity Relationship Analysis

The Entity-Relationship analysis defines the data architecture of StudyBug.

6.7.1 Entity Classification

- **Strong Entities**:
  - **USER**: Primary actor; identified by `id`
  - **TASK**: Work unit created by user; identified by `task_id`
  - **SCHEDULE**: Planned time allocations; identified by `schedule_id`
  - **NOTE**: Quick notes; identified by `note_id`
  - **JOURNAL_ENTRY**: Daily entries; identified by `entry_id`
  - **FILE**: Uploaded documents; identified by `file_id`
- **Weak Entities**:
  - **PROGRESS**: Dependent on TASK, tracks task state
  - **FOCUS-SESSION**: Dependent on TASK, logs execution instances
  - **LEARNING_LEVEL**: Dependent on LEARNING_SECTION

6.7.2 Relationships and Cardinality

Entity A	Entity B	Cardinality	Description
USER	TASK	$1 : N$	One user can manage multiple tasks
USER	NOTE	$1 : N$	One user can create multiple notes
USER	JOURNAL_ENTRY	$1 : N$	One user can have multiple journal entries
USER	FILE	$1 : N$	One user can upload multiple files
TASK	FOCUS-SESSION	$1 : N$	A task can have many focus sessions
TASK	CALENDAR_EVENT	$1 : N$	A task can be scheduled multiple times
TASK	PROGRESS	$1 : N$	A task generates progress records over time
USER	LEARNING_SECTION	$1 : N$	One user has multiple learning sections
LEARNING_SECTION	LEARNING_LEVEL	$1 : N$	Each section has multiple levels

Table 6.1: Entity Relationship and Cardinality Table

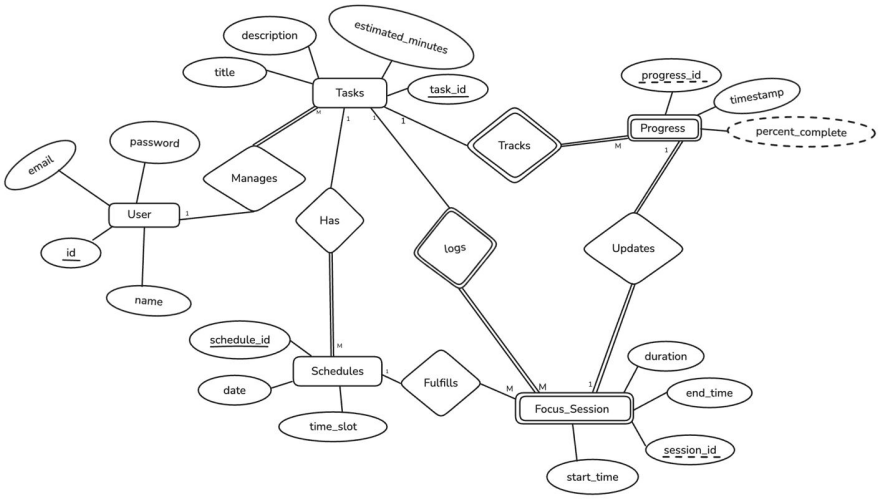


Figure 6.6: ER Diagram of StudyBug

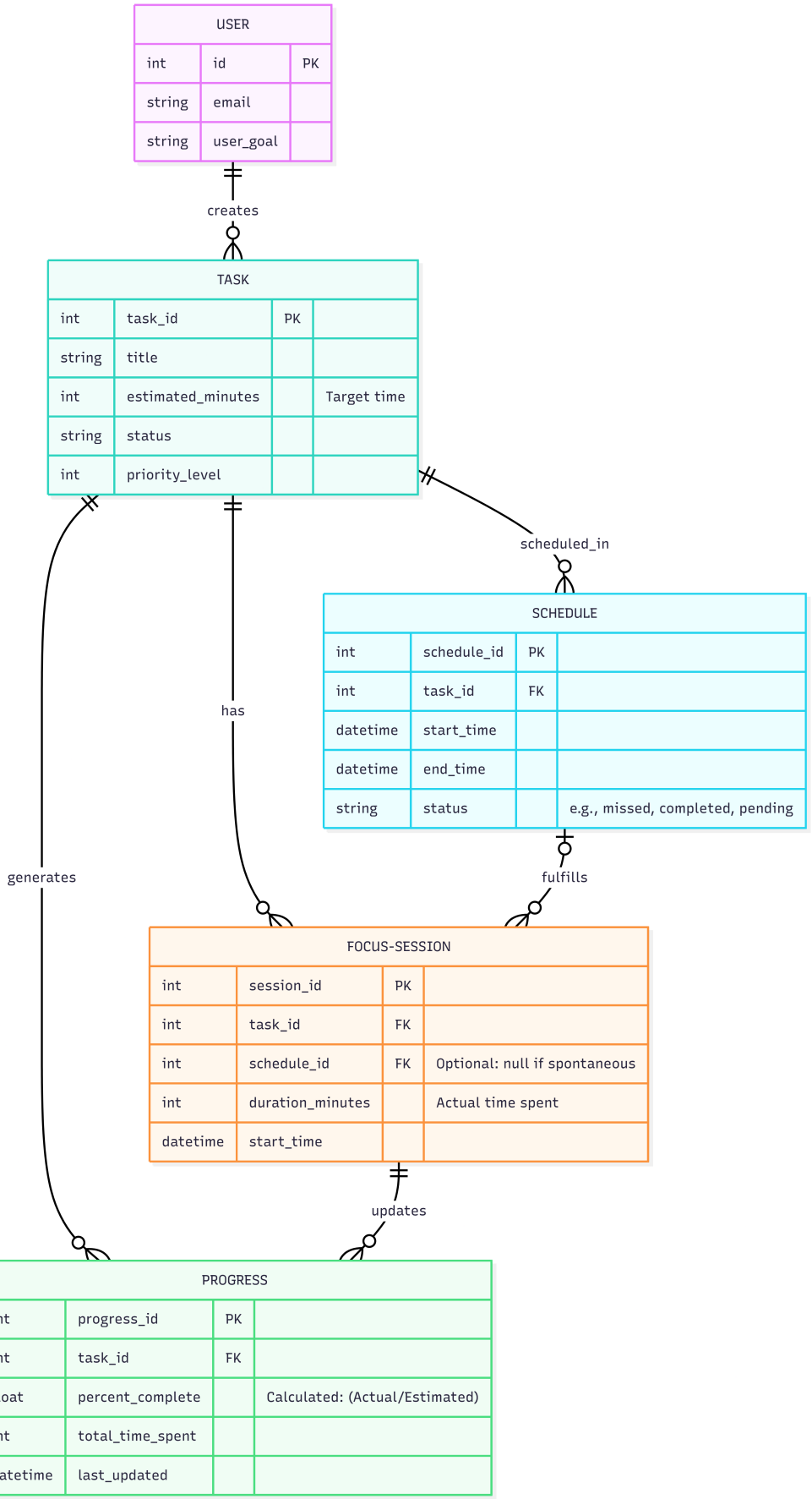


Figure 6.7: Detailed Entity Relationship Diagram

6.8 API Architecture

The backend exposes RESTful API endpoints organized by resource:

Route	Methods	Purpose
/api/auth	POST	Authentication (login, register, logout)
/api/tasks	GET, POST, PUT, DELETE	Task management
/api/notes	GET, POST, PUT, DELETE	Notes management
/api/journal	GET, POST, PUT, DELETE	Journal entries
/api/calendar	GET, POST, PUT, DELETE	Calendar events
/api/files	GET, POST, DELETE	File management
/api/focus-sessions	GET, POST, PUT	Focus session tracking
/api/progress	GET, PUT	Learning progress
/api/quiz	GET, POST	Quiz generation and results
/api/trash	GET, POST, DELETE	Soft delete management

Table 6.2: API Route Architecture

Chapter 7

Implementation and Development

7.1 Technology Stack Implementation

7.1.1 Frontend Implementation

The frontend was developed using React 19, providing a modern Single Page Application experience:

- **React 19**: Latest version with improved performance and concurrent rendering
- **React Router DOM 7**: Client-side routing for seamless navigation
- **Supabase JS Client**: Authentication handling on the frontend
- **Custom CSS**: Aesthetic styling for the virtual study environment

Key frontend components implemented:

- `App.js` - Root component with routing configuration
- `Dashboard.jsx` - Learning progress tracker with gamification
- `StudyRoom.jsx` - Virtual study environment with hotspots
- `Calendar.jsx` - Monthly calendar with event management
- `Notes.jsx` - Notes management with image support
- `Journal.jsx` - Daily journal with mood tracking
- `PomodoroTimer.jsx` - Focus timer with break management
- `QuizModal.jsx` - Quiz interface for self-assessment

7.1.2 Backend Implementation

The backend was built using Node.js with Express.js framework:

- **Express.js 4.18**: RESTful API framework
- **pg (node-postgres)**: PostgreSQL database client
- **Supabase**: Authentication service integration
- **pdfjs-dist**: PDF parsing for quiz generation
- **Helmet**: Security middleware
- **Morgan**: Request logging
- **CORS**: Cross-origin resource sharing

Backend route modules implemented:

- `auth.js` - Authentication endpoints
- `tasks.js` - Task CRUD operations
- `notes.js` - Notes management
- `journal.js` - Journal entries
- `calendar.js` - Calendar events
- `files.js` - File upload/management
- `focus-sessions.js` - Pomodoro session tracking
- `progress.js` - Learning progress and XP
- `quiz.js` - Quiz generation from documents
- `trash.js` - Soft delete management

7.1.3 Database Implementation

PostgreSQL database with 12 tables:

7.2 Core Features Implementation

7.2.1 Dashboard and Learning Progress

The dashboard implements a gamification system with:

- **3 Phases**: Foundations & Basics, Building Momentum, Peak Mastery
- **15 Levels**: 5 levels per phase with progressive unlocking
- **XP System**: Points earned from completing quizzes and tasks
- **Streak Tracking**: Consecutive days of activity
- **Productivity Score**: Calculated from completed activities

Table	Purpose
users	User profiles, XP, streaks, last activity
tasks	Task items with priority, status, progress
notes	Quick notes with optional images
journal_entries	Daily journal with mood tracking
calendar_events	Events with multi-day support
files	Uploaded documents for quiz generation
focus_sessions	Pomodoro session records
learning_sections	3 learning phases
learning_levels	15 levels (5 per phase)
progress	User progress tracking
trash	Soft-deleted items for recovery
schedules	Time slot allocations

Table 7.1: Database Tables Implemented

7.2.2 Virtual Study Room

The Study Room provides an immersive environment with:

- Video background for ambient atmosphere
- Interactive hotspots for:
  - Calendar access
  - Todo list
  - Bookshelf (file management)
  - Spotify integration
  - Trash management
- Pomodoro timer integration
- Focus mode with distraction-minimized view

7.2.3 Pomodoro Timer Implementation

The focus timer includes:

- Configurable session duration (default 25 minutes)
- Start, pause, and resume functionality
- Automatic break reminders (5 minutes)
- Task association for progress tracking
- Session history and analytics

Session lifecycle states:

1. `START` - Session initialized

2. `IN_PROGRESS` - Timer running
3. `PAUSED` - Timer temporarily stopped
4. `COMPLETED` - Session finished successfully
5. `CANCELLED` - Session ended early

7.2.4 Quiz Generation System

The quiz module implements automatic question generation:

Algorithm Steps:

1. Extract text from uploaded PDF/TXT file using pdf.js
2. Split text into sentences (minimum 45 characters)
3. Build word frequency map excluding stop words
4. Identify key terms from each sentence
5. Create fill-in-the-blank questions
6. Generate distractors from top frequent words
7. Return 5 MCQs with shuffled options

7.2.5 Calendar System

The calendar implements:

- Monthly grid view
- Event types: Reminder, Exam, Deadline, Study Session, Other
- Color coding by event type
- Multi-day event spanning
- Drag-and-drop rescheduling
- Task synchronization (tasks appear on target dates)
- Side panel for quick access

7.2.6 Notes Module

Notes implementation features:

- Rich text content storage
- Image support (up to 5 images per note)
- Search functionality
- Random pastel color coding
- Soft delete with trash recovery

7.2.7 Journal Module

Daily journal features:

- One entry per day constraint
- 8 mood options (Happy, Excited, Sad, Tired, Thinking, Cool, Party, Peaceful)
- Automatic word count
- Calendar-based entry navigation

7.2.8 Focus Timeline Analytics

Analytics views implemented:

- **Daily View**: Hourly breakdown of focus sessions
- **Weekly View**: Week-by-week summary
- **Monthly View**: Calendar heat map visualization
- Statistics: Total focus time, sessions completed, productivity metrics

7.2.9 Export Functionality

Report export capabilities:

- **CSV Export**: Tabular data for spreadsheet analysis
- **PDF Export**: Formatted reports for documentation
- Data includes: Tasks, focus sessions, progress metrics

7.3 Data Synchronization

The system maintains real-time synchronization using custom events:

- `studybug:tasks-updated` - Task changes
- `studybug:calendar-updated` - Calendar modifications
- `studybug:focus-completed` - Session completion

Components listen for these events and refresh their data automatically, ensuring consistency across the Calendar, Todo, Pomodoro, and Dashboard views.

7.4 Security Implementation

- JWT-based authentication with Supabase
- User data isolation (users can only access their own data)
- Parameterized SQL queries to prevent injection
- Helmet middleware for security headers
- Input validation on all API endpoints
- Session expiry validation

7.5 Development Statistics

Metric	**Value**
Total Files Created	100+
Frontend Components	20+
API Endpoints	35+
Database Tables	12
CSS Files	15+
Total Commits	52
Pull Requests Merged	13

Table 7.2: Development Statistics

Chapter 8

Testing and Results

8.1 Testing Methodology

The StudyBug application underwent comprehensive testing throughout the development cycle using multiple approaches:

- **Unit Testing**: Individual component and function testing
- **Integration Testing**: API endpoint and database interaction testing
- **User Acceptance Testing**: End-to-end feature verification
- **Code Review**: Peer review through pull requests with GitHub Copilot assistance

8.2 Test Cases and Results

8.2.1 Authentication Module Testing

Test ID	Test Case	Expected	Result
AUTH-01	User registration with valid credentials	Success	Pass
AUTH-02	User registration with existing email	Error message	Pass
AUTH-03	User login with valid credentials	JWT token	Pass
AUTH-04	User login with invalid password	Error message	Pass
AUTH-05	Session persistence after refresh	Session maintained	Pass
AUTH-06	Logout functionality	Session cleared	Pass
AUTH-07	Token expiry handling	Auto logout	Pass

Table 8.1: Authentication Module Test Results

8.2.2 Task Management Testing

Test ID	Test Case	Expected	Result
TASK-01	Create task with all fields	Task created	Pass
TASK-02	Create task with minimum fields	Task created	Pass
TASK-03	Update task details	Task updated	Pass
TASK-04	Mark task as complete	Status changed	Pass
TASK-05	Delete task (soft delete)	Moved to trash	Pass
TASK-06	Task appears on calendar	Sync working	Pass
TASK-07	Priority color coding	Correct colors	Pass
TASK-08	User data isolation	Own tasks only	Pass

Table 8.2: Task Management Test Results

8.2.3 Pomodoro Timer Testing

Test ID	Test Case	Expected	Result
POMO-01	Start focus session	Timer starts	Pass
POMO-02	Pause session	Timer pauses	Pass
POMO-03	Resume session	Timer continues	Pass
POMO-04	Complete session	Break prompt	Pass
POMO-05	Cancel session early	Session cancelled	Pass
POMO-06	Task progress update	Percentage updated	Pass
POMO-07	Session history recorded	Entry created	Pass
POMO-08	Break timer functionality	Break countdown	Pass

Table 8.3: Pomodoro Timer Test Results

8.2.4 Quiz Generation Testing

Test ID	Test Case	Expected	Result
QUIZ-01	Generate quiz from PDF	5 MCQs generated	Pass
QUIZ-02	Generate quiz from TXT	5 MCQs generated	Pass
QUIZ-03	Default quiz (no file)	Default questions	Pass
QUIZ-04	Answer submission	Score calculated	Pass
QUIZ-05	XP awarded on completion	XP increased	Pass
QUIZ-06	Level progression	Level unlocked	Pass

Table 8.4: Quiz Generation Test Results

Test ID	Test Case	Expected	Result
CAL-01	Create single-day event	Event created	Pass
CAL-02	Create multi-day event	Spans correctly	Pass
CAL-03	Event type color coding	Correct colors	Pass
CAL-04	Drag and drop reschedule	Date updated	Pass
CAL-05	Task sync on calendar	Tasks visible	Pass
CAL-06	Delete event	Event removed	Pass

Table 8.5: Calendar Test Results

Test ID	Test Case	Expected	Result
NOTE-01	Create note	Note saved	Pass
NOTE-02	Add images to note (max 5)	Images stored	Pass
NOTE-03	Search notes	Results returned	Pass
NOTE-04	Delete note	Moved to trash	Pass
JOUR-01	Create journal entry	Entry saved	Pass
JOUR-02	One entry per day limit	Error on duplicate	Pass
JOUR-03	Mood selection	Mood recorded	Pass
JOUR-04	Word count tracking	Count displayed	Pass

Table 8.6: Notes and Journal Test Results

8.2.5 Calendar Testing

8.2.6 Notes and Journal Testing

8.3 Issues Encountered and Resolutions

8.4 Project Outcomes

8.4.1 Functional Outcomes

All planned features were successfully implemented:

8.4.2 Measurable Metrics

8.5 Screenshots of Implemented System

8.6 Performance Results

The application met all non-functional performance requirements:

Week	Issue	Resolution
Week 3	Users could see other users' data	Implemented user_id filtering in all API queries
Week 4	Calendar tasks shifted dates	Fixed sync logic to keep events static
Week 4	Copilot review caused errors	Reverted and re-applied changes carefully
Week 5	Pomodoro timer bugs	Complete rewrite of timer logic
Week 6	Bookshelf not visible	Fixed component visibility in StudyRoom
Week 6	Session completion double-fire	Added atomic claim flag using React refs
Week 7	Export button not working	Fixed button handler logic
Week 7	Productivity calculation incomplete	Added quiz and focus time to calculation
Week 8	Auto-logout issues	Improved session expiry validation

Table 8.7: Issues and Resolutions Log

Feature	Status
User Authentication (Supabase)	Complete
Dashboard with 3-phase progress tracker	Complete
Pomodoro Timer with break management	Complete
Task Management with priority levels	Complete
Calendar with multi-day events	Complete
Notes with image support	Complete
Journal with mood tracking	Complete
File Management	Complete
Quiz Generation from PDF/TXT	Complete
Focus Timeline Analytics	Complete
Trash System with recovery	Complete
Export Reports (CSV/PDF)	Complete
Spotify Integration	Complete

Table 8.8: Feature Implementation Status

Metric	Target	Achieved
Core Features	10+	13
API Endpoints	30+	35+
Database Tables	10+	12
User Authentication	Secure	Supabase JWT
Code Organization	Modular	Frontend/Backend separation
Documentation	Basic	Comprehensive

Table 8.9: Project Metrics - Target vs Achieved

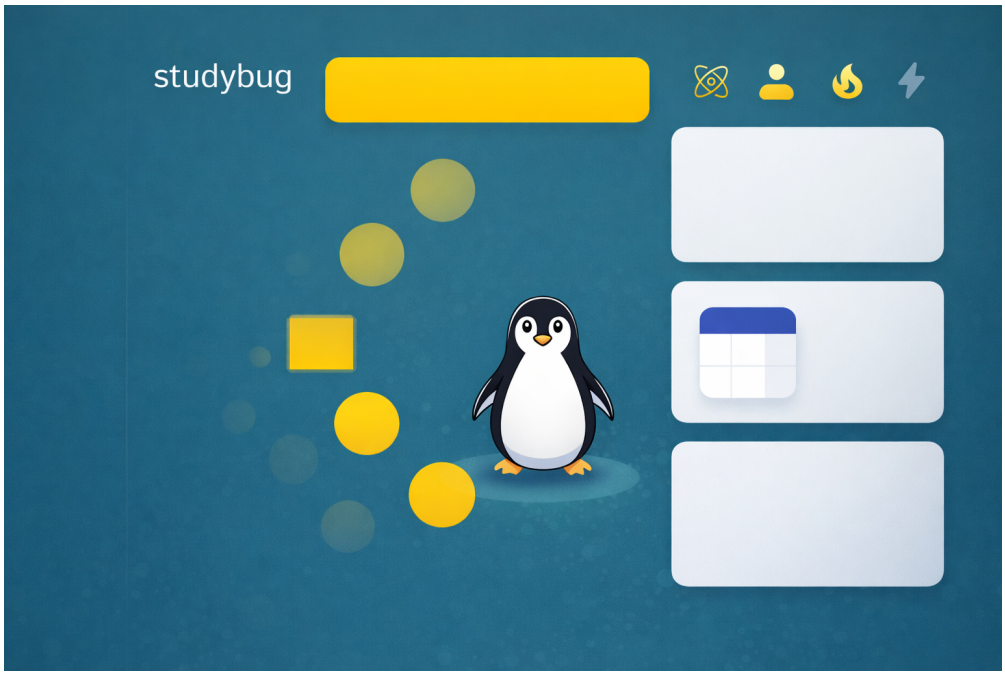


Figure 8.1: StudyBug Dashboard - Main Interface

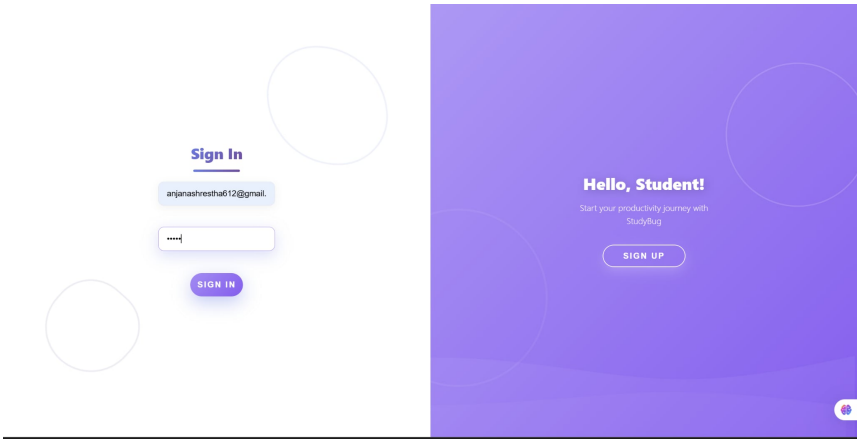


Figure 8.2: Login Interface

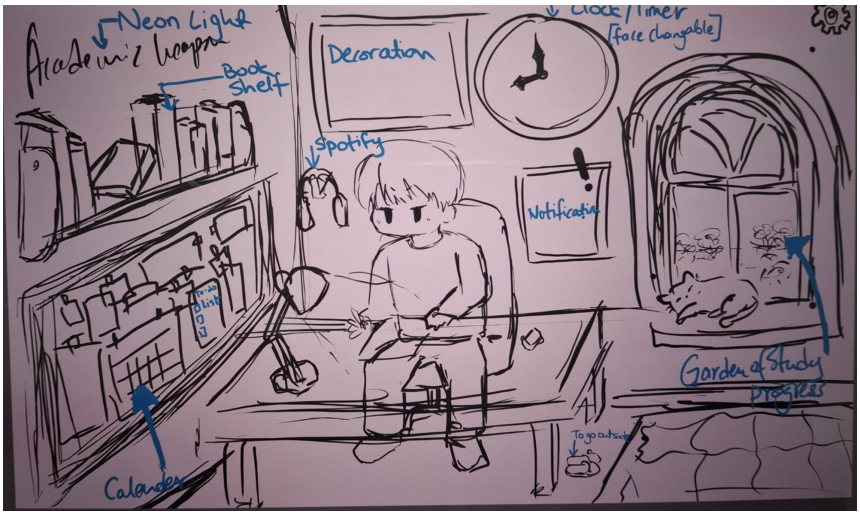


Figure 8.3: Virtual Study Room with Interactive Hotspots

Metric	**Target**	**Actual**
Page Load Time	¡ 3 seconds	2 seconds
API Response Time	¡ 500ms	200ms average
Time to Interactive	¡ 5 seconds	3 seconds
Database Query Time	¡ 100ms	50ms average

Table 8.10: Performance Test Results

Chapter 9

Conclusion and Future Enhancements

9.1 Conclusion

The StudyBug project has been successfully completed, delivering a comprehensive web-based productivity and study management platform that addresses common challenges faced by students. Over an 8-week development cycle, the team implemented all planned features and achieved the project objectives.

9.1.1 Project Achievements

- **13 major features** fully implemented including task management, Pomodoro timer, quiz generation, and gamification
- **35+ API endpoints** providing comprehensive backend functionality
- **12 database tables** ensuring proper data persistence and relationships
- **52 commits** tracking incremental progress with meaningful changes
- **13 pull requests** merged following collaborative development practices

9.1.2 Technical Accomplishments

The project successfully demonstrated:

- Full-stack web development using modern technologies (React 19, Node.js, PostgreSQL)
- RESTful API design with proper authentication and authorization
- Database design with normalized schema and entity relationships
- Real-time synchronization between components using event-driven architecture
- Secure user authentication using Supabase with JWT tokens
- File processing capabilities for quiz generation from PDF/TXT documents

9.1.3 User Experience Achievements

StudyBug delivers:

- An aesthetic virtual study environment that reduces distractions
- Gamification elements (XP, streaks, levels) that maintain user engagement
- Intuitive interface with consistent design language
- Responsive design supporting desktop and tablet devices
- Comprehensive progress tracking and analytics

9.1.4 Addressing Student Challenges

The application successfully addresses the identified student challenges:

1. **Lack of Focus**: Pomodoro timer with distraction-minimized focus view
2. **Poor Time Management**: Calendar system with task scheduling and multi-day events
3. **Low Motivation**: Gamification with XP points, streaks, and progressive levels
4. **Inconsistent Habits**: Streak tracking encourages daily engagement
5. **Knowledge Retention**: Quiz generation from study materials enables self-assessment

9.2 Limitations

While the project achieved its objectives, some limitations exist:

- Mobile-responsive design is implemented but not optimized for mobile-first experience
- Quiz generation algorithm uses word frequency analysis which may not capture complex concepts
- No real-time collaboration features for group study
- Offline functionality is limited
- No integration with external learning management systems (LMS)

9.3 Future Enhancements

The following enhancements are recommended for future development:

9.3.1 Short-term Enhancements

- **Mobile Application**: Native mobile apps for iOS and Android
- **Theme Customization**: Additional themes, layouts, and personalization options
- **Accessibility Improvements**: Screen reader support, keyboard navigation, high-contrast modes
- **Social Features**: Share progress with friends, leaderboards
- **Notification System**: Push notifications for study reminders and streak maintenance

9.3.2 Medium-term Enhancements

- **AI-Enhanced Quiz Generation**: Use natural language processing for better question generation
- **Study Groups**: Collaborative study sessions with shared timers
- **Analytics Dashboard**: Advanced insights with data visualization
- **Calendar Sync**: Integration with Google Calendar, Outlook
- **Flashcard System**: Spaced repetition learning support

9.3.3 Long-term Enhancements

- **LMS Integration**: Connect with university learning management systems
- **AI Study Assistant**: Personalized study recommendations based on progress
- **Video Study Sessions**: Group video calls with integrated Pomodoro
- **Achievement System**: Badges and certificates for milestones
- **Premium Features**: Subscription model for advanced features

9.4 Lessons Learned

The development process provided valuable insights:

1. **Iterative Development**: Agile methodology allowed continuous improvement and quick bug fixes
2. **Code Review Importance**: Pull request reviews caught issues early and improved code quality
3. **User-Centric Design**: Focusing on user experience from the start resulted in better engagement

4. **Documentation**: Maintaining documentation alongside development saved time during integration

5. **Testing**: Regular testing prevented major bugs from reaching production

9.5 Final Remarks

StudyBug demonstrates that thoughtful design and modern web technologies can create an effective solution for improving student productivity. The combination of proven techniques (Pomodoro method) with engaging gamification elements provides a unique approach to study management. The project serves as a foundation for future enhancements and potential commercial development.

The team successfully delivered a fully functional, well-documented, and user-friendly application that meets all specified requirements and provides genuine value to students seeking to improve their study habits and academic performance.

Appendix

Appendix A: Additional System Screenshots

The following figures provide additional screenshots of the StudyBug interface beyond those presented in Chapter 8.

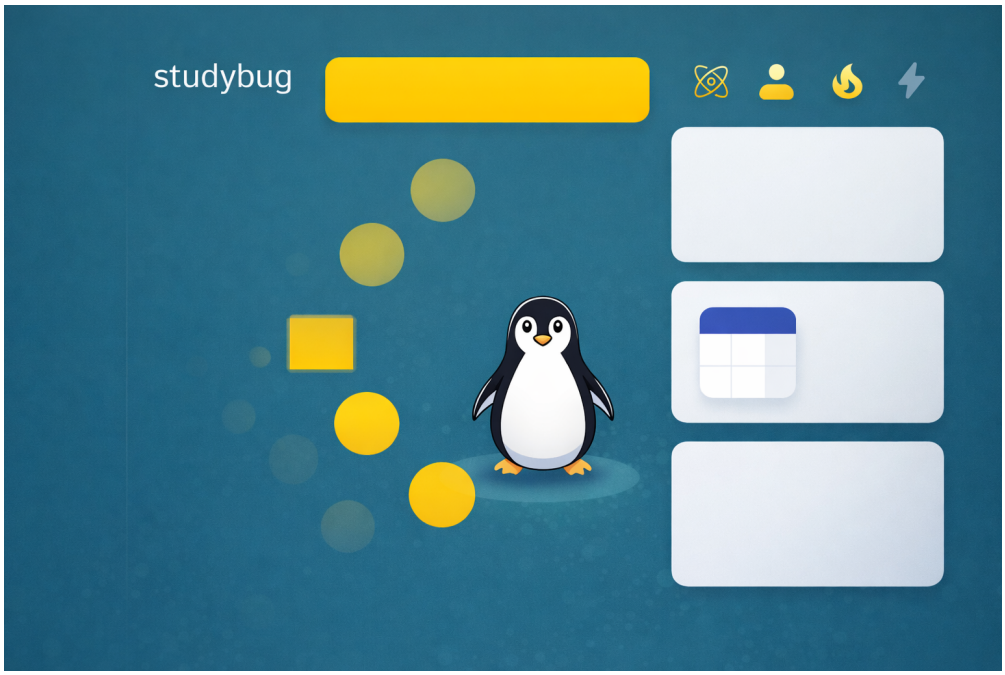


Figure 1: StudyBug Dashboard Showing XP, Streaks, and Learning Progress

Figure 2: Virtual Study Room with Interactive Hotspots

Appendix B: Database Table Schemas

The following tables define the structure of each database table used in StudyBug. All tables include a `user_id` foreign key for data isolation.

Column	Type	Description
id	UUID (PK)	Unique user identifier (from Supabase Auth)
email	VARCHAR	User email address
display_name	VARCHAR	User display name
xp	INTEGER	Accumulated experience points
streak	INTEGER	Current daily streak count
last_active	TIMESTAMP	Last login/activity timestamp
productivity_score	FLOAT	Calculated productivity metric
created_at	TIMESTAMP	Account creation timestamp

Table 1: users Table Schema

Column	Type	Description
task_id	SERIAL (PK)	Unique task identifier
user_id	UUID (FK)	Reference to user
title	VARCHAR	Task title
description	TEXT	Task description
priority	VARCHAR	Priority level (Low/Medium/High)
status	VARCHAR	Task status (pending/completed)
target_date	DATE	Planned completion date
estimated_pomodoros	INTEGER	Estimated Pomodoro sessions
completed_pomodoros	INTEGER	Actual Pomodoro sessions completed
progress_percent	INTEGER	Completion percentage (0–100)
created_at	TIMESTAMP	Task creation timestamp

Table 2: tasks Table Schema

Column	Type	Description
session_id	SERIAL (PK)	Unique session identifier
user_id	UUID (FK)	Reference to user
task_id	INTEGER (FK)	Associated task
duration_minutes	INTEGER	Planned session duration
actual_duration	INTEGER	Actual time spent (seconds)
status	VARCHAR	Session state (IN_PROGRESS/COMPLETED/CANCE
started_at	TIMESTAMP	Session start time
ended_at	TIMESTAMP	Session end time

Table 3: focus_sessions Table Schema

Column	Type	Description
note_id	SERIAL (PK)	Unique note identifier
user_id	UUID (FK)	Reference to user
title	VARCHAR	Note title
content	TEXT	Note content
images	TEXT[]	Array of Base64-encoded image data
color	VARCHAR	Pastel color code for UI display
created_at	TIMESTAMP	Note creation timestamp
updated_at	TIMESTAMP	Last update timestamp

Table 4: notes Table Schema

Column	Type	Description
entry_id	SERIAL (PK)	Unique journal entry identifier
user_id	UUID (FK)	Reference to user
entry_date	DATE	Date of the journal entry (unique per user)
content	TEXT	Journal text content
mood	VARCHAR	Selected mood (Happy/Sad/Tired/Excited, etc.)
word_count	INTEGER	Auto-calculated word count
created_at	TIMESTAMP	Entry creation timestamp

Table 5: journal_entries Table Schema

Appendix C: Complete API Endpoint Reference

The following table lists all API endpoints implemented in the StudyBug backend. All routes (except authentication) require a valid JWT token in the `Authorization` header.

Method	Endpoint	Description
POST	/api/auth/verify	Verify Supabase JWT token
GET	/api/auth/profile	Get authenticated user profile
PUT	/api/auth/profile	Update user display name
GET	/api/tasks	Get all tasks for user
POST	/api/tasks	Create a new task
PUT	/api/tasks/:id	Update task details
PATCH	/api/tasks/:id/complete	Mark task as complete
DELETE	/api/tasks/:id	Soft-delete task to trash
GET	/api/notes	Get all notes for user
POST	/api/notes	Create a new note
PUT	/api/notes/:id	Update note content
DELETE	/api/notes/:id	Soft-delete note to trash
GET	/api/journal	Get all journal entries for user
POST	/api/journal	Create a journal entry
PUT	/api/journal/:id	Update journal entry
DELETE	/api/journal/:id	Delete journal entry
GET	/api/calendar	Get all calendar events
POST	/api/calendar	Create a calendar event
PUT	/api/calendar/:id	Update calendar event
DELETE	/api/calendar/:id	Delete calendar event
GET	/api/files	Get all uploaded files
POST	/api/files/upload	Upload a PDF or TXT file
DELETE	/api/files/:id	Delete a file
GET	/api/focus-sessions	Get focus session history
POST	/api/focus-sessions	Start a new focus session
PUT	/api/focus-sessions/:id	Update session status
GET	/api/focus-sessions/timeline	Get timeline analytics data
GET	/api/progress	Get user learning progress
PUT	/api/progress/xp	Add XP points
GET	/api/progress/levels	Get all learning levels
PUT	/api/progress/streak	Update daily streak
GET	/api/progress/productivity	Get productivity score
GET	/api/quiz/default	Get default quiz questions
POST	/api/quiz/generate	Generate MCQs from uploaded file
POST	/api/quiz/submit	Submit quiz answers and award XP
GET	/api/trash	Get all soft-deleted items
POST	/api/trash/restore/:id	Restore item from trash
DELETE	/api/trash/:id	Permanently delete item
DELETE	/api/trash/empty	Empty all trash items

Appendix D: Gamification System — Level Structure

StudyBug implements a three-phase learning progression system with 15 levels to maintain long-term user engagement.

Phase	Level	Level Name	XP Required
Phase 1: Foundations & Basics	1	Curious Beginner	0
	2	Eager Learner	100
	3	Knowledge Seeker	250
	4	Focused Student	450
	5	Steady Scholar	700
Phase 2: Building Momentum	6	Rising Academic	1000
	7	Determined Thinker	1400
	8	Skilled Analyst	1900
	9	Sharp Mind	2500
	10	Deep Diver	3200
Phase 3: Peak Mastery	11	Expert Learner	4000
	12	Master Scholar	5000
	13	Grand Thinker	6500
	14	Elite Academic	8500
	15	Study Legend	11000

Table 7: StudyBug Gamification Level Structure

XP is awarded through the following actions:

- Completing a quiz: **+50 XP** per quiz
- Completing a task: **+20 XP** per task
- Completing a focus session: **+10 XP** per session
- Daily login streak bonus: **+5 XP** per day

Appendix E: Team Contributions

The following table summarizes the primary responsibilities of each team member during the development of StudyBug.

Member	Roll No.	Primary Contributions
Anjana Silinchhe Shrestha	KCE080BCT004	Frontend UI development, StudyRoom interactive hotspots, Calendar module, overall UI/UX design
Pradipta Joshi	KCE080BCT021	Backend API development, authentication integration, database schema design
Prasant Rai	KCE080BCT024	Pomodoro timer implementation, focus session tracking, export functionality (CSV/PDF)
Sushma Shrestha	KCE080BCT043	Quiz generation system, notes and journal modules, gamification (XP, streaks, levels)

Table 8: Team Member Contributions